

Práctica 1: Implementación y Análisis de Bloques en GNU Radio para Acumulación y Diferenciación de Señales

Carlos David Cuadros Sanabria - 2201524

Carlos Julio Solano Cala - 2204549

Kamila Andrea Contreras Amaya - 2182782

https://github.com/AndreaContre14/CommII_A1_G4.git

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

Abstract

This paper presents the implementation and validation of custom blocks in GNU Radio using Python. Accumulator and Differentiator blocks were developed to extract deterministic (amplitude, frequency, phase, period) and probabilistic (mean, variance, RMS) parameters from signals, allowing real-time analysis and improved noise handling. The results demonstrate the effectiveness of the approach and lay the foundation for future optimizations in software-defined radio systems.

1 Introducción

Para la correcta construcción de un sistema de comunicación, es esencial comprender las diversas características de la señal, lo que permite realizar los tratamientos adecuados y mejorar la eficiencia del sistema. Estas características se conocen como parámetros fundamentales, entre estos se encuentran la amplitud, frecuencia, fase y período. Cada uno de estos factores influye directamente en el comportamiento y rendimiento del sistema de comunicación [1].

Sin embargo, estos parámetros fundamentales son más fáciles de identificar en señales determinísticas, es decir, aquellas que se mantienen consistentes y predecibles a lo largo del tiempo, siendo de esta forma predecibles. En su contraparte tendremos las señales aleatorias, requieren de diversos procesos para ser captados, ya que como su nombre lo indica son impredecibles. Una solución óptima es trabajar con parámetros probabilísticos, permitiéndonos tener un rango más claro sobre el comportamiento de la señal y realizar aproximaciones

precisas de sus características [2].

Los parámetros probabilísticos que se utilizan

incluyen la varianza, la media, la media cuadrática (M.C.), la potencia promedia, el valor RMS y desviación estándar (D.E.). Debido a la naturaleza variable de la señal, es necesario analizar la señal en segmentos de tiempos pequeños. Al hacerlo, se pueden calcular estadísticas locales que reflejan el comportamiento de la señal con mayor precisión, que, si analizáramos intervalos mucho más grandes, a esto se le conoce como la teoría de las señales aleatorias [2].

Para desarrollar sistemas de comunicación avanzados, es posible implementar parámetros probabilísticos mediante el uso de bloques programados en Python dentro del entorno de GNU Radio. Esta capacidad de programación ofrece una flexibilidad mucho mayor en comparación con los bloques predefinidos del software, ya que permite personalizar y ajustar parámetros de manera específica según las necesidades del proyecto. Esto no solo facilita la optimización del rendimiento del sistema, sino que también permite la integración de algoritmos complejos, el procesamiento de señales en tiempo real y el control preciso de aspectos críticos, como el filtrado de señales. Al aprovechar la programación en Python, se pueden diseñar soluciones más robustas y adaptadas a escenarios específicos, lo que resulta fundamental en aplicaciones modernas de telecomunicaciones [1].

2 Procedimiento

Inicialmente, se crea una rama para el desarrollo de la práctica desde el repositorio, denominada Práctica_1, la cual debe ser compartida con los demás integrantes del grupo. A esta rama se

le asigna un repositorio con el mismo nombre, y dentro de dicho repositorio se crean dos carpetas: GNU Radio e Informe. Cada integrante del grupo establece su propia rama individual dentro de la rama Practica_1, nombrada como P1_Nombre1, P1_Nombre2, y P1_Nombre3. Se accede a estas ramas con el comando "git checkout P1_Nombre

- Se implementaron dos bloques de Python en el entorno de GNU-Radio: un acumulador y un diferenciador, con ayuda del libro guía propuesto en la práctica, en la sección 1.2.0.1 [1], se obtuvieron los códigos guías para implementarlos en los respectivos bloques.

• Bloque acumulador

El bloque de Python denominado Accumulator se programó para sumar de manera consecutiva las muestras de la señal de entrada según la ecuación:

$$y[n] = y[n-1] + x[n]$$

Para evitar que la salida permaneciera cercana a cero con una señal cuadrada simétrica, se aplicó un desplazamiento de -0.5 a la fuente. Este offset impide la cancelación entre valores positivos y negativos, lo que permite apreciar de manera clara la evolución acumulada. Finalmente, se conectó la salida del bloque y la señal de entrada a un visualizador en tiempo real, a fin de comparar la forma de onda original y la forma procesada.

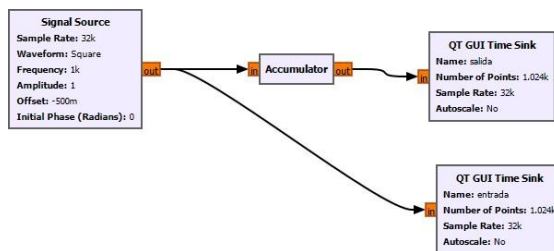


Fig. 2.1. Diagrama de flujo en GNU Radio que muestra un bloque acumulador con una señal cuadrada de entrada y un offset de -0.5.

• Bloque diferenciador

El bloque diferenciador es aquel que realiza la operación de la diferenciación, es decir, calcula la derivada con respecto al tiempo, de manera matemática se puede ver de la siguiente manera.

$$y[n] = x[n] - x[n-1]$$

Nuevamente se emplea un bloque de Python en GNURadio para implementar un algoritmo que me haga esta operación, en donde se le ingresa una señal con el fin de obtener su derivada.

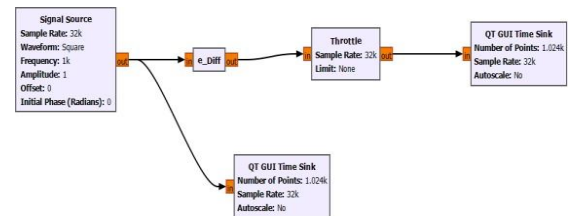


Fig.2.2 Implementación del bloque diferenciador

A continuación, se debe crear un tercer bloque para mostrar las estadísticas de una señal. Este bloque, al igual que los anteriores, se implementa utilizando el bloque 'Python block' en GNU Radio, donde se realiza con base al libro guía sección 1.2.0.1 [1]. A partir de esta implementación se debe realizar una aplicación, en este caso disminuir la afectación del ruido Gaussiano en la estadística, para ello se divide la señal en 10 ventanas de tamaño 4096 muestras y se decide utilizar funciones propias de Python para intentar comparar con las sugeridas por el libro.

• Bloque promedio del tiempo

El bloque desarrollado, llamado Promedios_de_tiempos, fue definido como un bloque de sincronización (gr.sync_block). Se inicializó con variables acumulativas para mantener un seguimiento del cálculo en tiempo real, evitando pérdida de información en flujos continuos de datos.

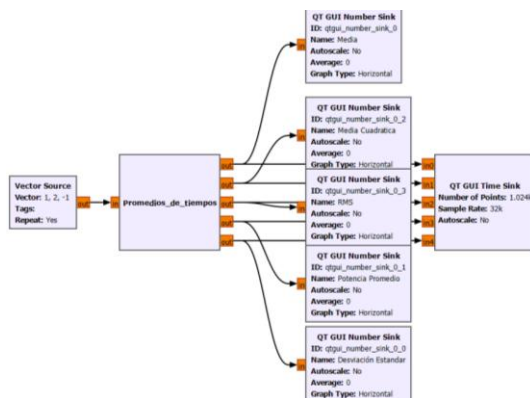


Fig.2.2 Diagrama de flujo en GNU Radio que muestra las estadísticas de una señal.

Los siguientes procedimientos fueron implementados dentro del método `work()`:

- **Media:** Se calcula acumulando la sumatoria de la señal y dividiéndola por la cantidad total de muestras recibidas.
- **Media cuadrática:** Similar a la media, pero utilizando los valores de la señal elevados al cuadrado.
- **Valor RMS:** Se obtiene tomando la raíz cuadrada de la media cuadrática.
- **Potencia promedio:** Se calcula elevando al cuadrado el valor RMS.
- **Desviación estándar:** Se obtiene mediante la sumatoria acumulada de la diferencia al cuadrado entre la señal y su media, dividiendo por el total de muestras y luego extrayendo la raíz cuadrada.

A partir de esta implementación se debe realizar una aplicación, en este caso disminuir la afectación del ruido Gaussiano en la estadística, para ello se divide la señal en 10 ventanas de tamaño 4096 muestras y se decide utilizar funciones propias de Python para intentar comparar con las sugeridas por el libro.

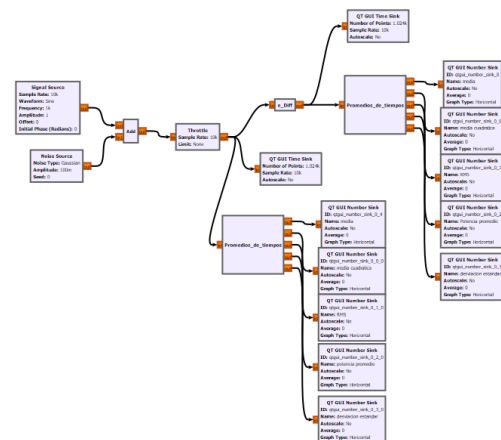


Fig.2.2 Implementación de la aplicación de reducción de ruido Gaussiano con las modificaciones realizadas.

Resultados

En la Fig. 3.1 se muestra la señal cuadrada de entrada (parte superior) con un desplazamiento de 0.5 y, en la parte inferior, la salida del bloque *Accumulator*. Se aprecia que la onda acumulada adquiere una forma triangular, ya que cada nivel constante de la señal cuadrada se integra de manera lineal mientras dure ese nivel. El offset introducido impide la cancelación entre valores positivos y negativos, lo que hace que la salida evolucione de forma apreciable y no permanezca cercana a cero.

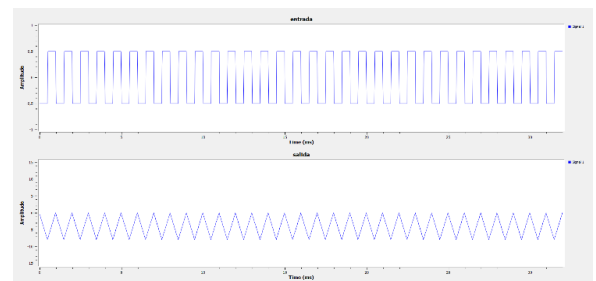


Fig. 3.1 Resultado de la señal de entrada y su versión acumulada.

- Para comprobar el correcto funcionamiento del bloque diferenciador Fig.2.2 se ingresan 2 tipos de señales, una cuadrada y una triangular. Se pudo observar como a la salida se logra obtener sus respectivas derivadas en el tiempo como lo podemos ver a continuación.

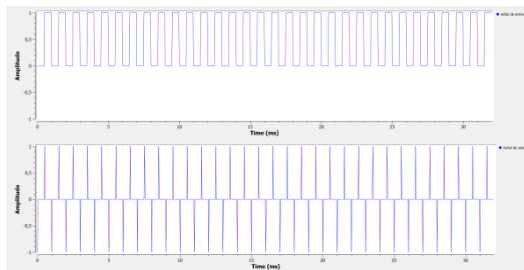


Fig.2.1 Derivación de señal cuadrada.

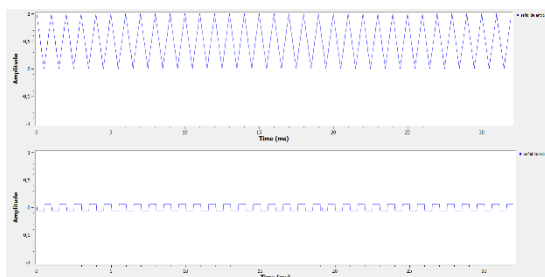


Fig.2.1 Derivación de señal triangular.

Calculando sus valores teóricos obtenemos:

Ruido	Media	Media cuadrada	RMS	Potencia promedio	Desviación estandar
0 (1)	0	0.5002	0.7073	0.5002	0.7073
0 (2)	0	0.5000	0.7071	0.5000	0.7071
0.25 (1)	0	0.5003	0.7073	0.5003	0.7073
0.25 (2)	0	0.5001	0.7071	0.5002	0.7072
0.05 (1)	0.0001	0.5028	0.7091	0.5028	0.7088
0.05 (2)	0	0.5008	0.7077	0.5008	0.7077
0.1 (1)	0	0.5113	0.7152	0.5112	0.7141
0.1 (2)	0.0002	0.5100	0.7138	0.5100	0.7137

Tabla 3.1 Resultados de los bloques de estadística (1) y aplicación (2) para diferentes niveles de ruido en una señal seno.

Conclusiones

La utilización de funciones propias de Python, en paralelo con

las sugeridas por el libro guía, permitió validar y comparar distintos métodos de reducción de ruido, confirmando la aplicabilidad de los conceptos teóricos aprendidos en clase.

Referente al desarrollo de los códigos propuestos se le hicieron pequeñas modificaciones como arreglar la indentación, cambiar unas comillas y corregir el orden de unas líneas de código para su correcto funcionamiento. En el caso del bloque acumulador y diferenciador si tenemos una señal que su media no es cero, y sus valores están sobre el eje positivo los valores de salida de los bloques tienden a infinito, pero en la realidad de la implementación llegan a un punto donde se desbordan y reinicia la operación como ocurría en sistemas digitales.

La implementación del bloque acumulador en GNU Radio, mediante Python, permitió sumar de manera consecutiva las muestras de la señal, ajustando el offset para evitar la cancelación de valores en señales cuadradas simétricas. Esto evidencia la importancia de realizar modificaciones adecuadas para visualizar correctamente la evolución de la señal.

Se comprende que las señales que salen de los bloques son señales tipo *stream*, esto afecta ya que el tratamiento de las señales tipo stream son diferentes a las de un vector, por lo cual tuvimos que reestructurar los códigos del libro guía como si las señales de salida fueran un vector con el fin de facilitar su tratamiento y que los bloques de GNU Radio produzcan el valor deseado a tiempo real.

Referencias

- [1] H. O. Boada and O. M. R. Torres, *Comunicaciones Digitales basadas en radio definida por software*, primera ed. Bucaramanga, Colombia: Editorial UIS, 2019.
- [2] S.-M. M. Yang, *Review of Signals and Systems, Signals and systems, and of Probability and Random Process*. Cham: Springer International Publishing, 2020. [Online]. Available: https://doi.org/10.1007/978-3-030-57706-3_9